

Paradigma Procedural ou **Modular**

Paradigmas de Linguagens de Programação

Ewerton Perdone Rangel, Pedro Henrique Rocha de Andrade

Instituto Federal Fluminense
Campus Bom Jesus do Itabapoana

30 de outubro de 2025

Sumário

1 Conceitos

2 Exemplos de Uso

3 Códigos

4 Quizz

Conceitos

Definição

Programação Procedural é um paradigma de programação classificado como programação imperativa, que envolve implementar o comportamento de um programa através de procedimentos (também conhecidos como funções, sub-rotinas) que chamam uns aos outros.

Características Principais

- Sequência de instruções executadas de cima para baixo
- Organização em procedimentos/funções
- Divisão de tarefas complexas em subtarefas menores
- Controle de fluxo através de estruturas sequenciais

Grandes Nomes

Outputs:

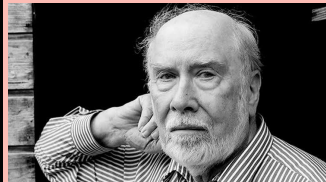
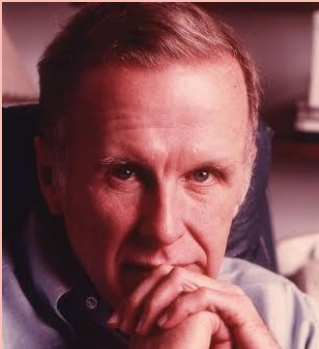


Figura 1: Esquerda: John Backus [1]; Meio: Dennis Ritchie [2]; Direita: Niklaus Wirth [3]

Contexto Histórico

Origens:

- **FORTRAN** (1957): Primeira linguagem procedural de alto nível, desenvolvida por John Backus na IBM para computação científica e matemática
- COBOL (1959): Desenvolvida pelo CODASYL para processamento de dados empresariais
- ALGOL (1958-1960): Influenciou o design de muitas linguagens modernas
- BASIC (1964): Criada para fins educacionais, tornando a programação mais acessível

Evolução:

- Pascal (1970): Criada por Niklaus Wirth para incentivar boas práticas de programação estruturada
- C (1972): Desenvolvida por Dennis Ritchie, tornou-se fundamental para programação de sistemas

Motivação

Objetivos principais:

- Decomposição de problemas: **Dividir problemas complexos em partes menores e gerenciáveis (procedimentos)**
- Reutilização de código: Através de funções e procedimentos bem definidos
- Manutenção: Código organizado em módulos facilita manutenção
- Eficiência: Controle direto sobre recursos do sistema
- Execução: Fluxo de execução claro e sequencial

Filosofia:

- 1 Foco em **como fazer** as coisas (procedimentos)
- 2 Dados e funções são tratados separadamente
- 3 Sequência de passos bem definida

Vantagens e Desvantagens

Vantagens

- **Simplicidade e facilidade de aprendizado para iniciantes**
- Eficiência na execução por evitar overhead de objetos e classes
- Controle explícito e previsível do fluxo do programa

Desvantagens

- Código pode crescer e se tornar difícil de manter em projetos grandes
- Reutilização limitada e falta de encapsulamento natural dos dados
- Dificuldade em modelar sistemas complexos com muitas interações de dados

Diferenças entre Procedimentos e Funções

- Procedimento: bloco que executa ações, mas **não retorna valor**
- Função: bloco que executa ações e possivelmente **retorna um valor resultante**

Exemplos de Uso

Aplicações Práticas

- 1 Programação de Sistemas: Desenvolvimento de sistemas operacionais (Linux, Windows kernel) e drivers de dispositivos
- 2 **Sistemas Embarcados**: Programação de microcontroladores, sistemas automotivos, dispositivos IoT
- 3 Computação Científica: Aplicações que exigem cálculos matemáticos intensivos e simulações
- 4 Aplicações de Alto Desempenho: Quando eficiência e controle de recursos são críticos
- 5 Sistemas Legados: Manutenção de grandes bases de código existentes
- 6 Automação e Scripts: Tarefas de administração de sistemas e processamento de dados

Exemplos de Uso

1. Programação de Sistemas:

- Kernel Linux (escrito em C)
- Componentes do Windows (C/C++)
- Drivers de hardware

2. Aplicações Científicas:

- Simulações físicas e matemáticas
- Análise de dados numéricos
- Modelagem computacional

3. Sistemas Críticos:

- Software aeroespacial (NASA usa C/FORTRAN)
- Sistemas automotivos embarcados
- Equipamentos médicos

4. Ferramentas de Desenvolvimento:

- Compiladores e interpretadores
- Ferramentas de processamento de texto
- Utilitários de sistema

Empresas que Utilizam

Exemplos

- Google [4];
- Microsoft [5];
- Amazon [6];
- Apple [7]

Integrações

Linguagens Multi-paradigma:

A programação procedural frequentemente coexiste com outros paradigmas:

- C++ Combina procedural, orientado a objetos e genérico
- **Python: Suporta procedural, OO e funcional**
- JavaScript: Procedural, OO e funcional

Integração com outros paradigmas:

- Com OO: Métodos em classes podem usar lógica procedural
- Com Funcional: Funções podem ser passadas como argumentos
- Com Declarativo: SQL dentro de código procedural

Integrações:

- **APIs C são o padrão para interoperabilidade entre linguagens**
- Bibliotecas procedurais (C) podem ser chamadas de outras linguagens

Códigos

Exemplo 1: Cotação Dólar

```
1 import requests
2
3 def buscar_cotacoes(base="USD"):
4     """Busca as cotações a partir de uma moeda base"""
5     url = f"https://api.exchangerate-api.com/v4/latest/{base}"
6     response = requests.get(url)
7     if response.status_code == 200:
8         return response.json()
9     else:
10        return None
11
12 def pegar_cotacao(data, moeda="BRL"):
13     """Extraí a cotação de uma moeda específica"""
14     if data and "rates" in data:
15         return data["rates"].get(moeda, None)
16     return None
17
18 def exibir_cotacao(valor, moeda="BRL"):
19     """Mostra a cotação formatada"""
20     if valor:
21         print(f"Cotação do dólar em {moeda}: {valor}")
22     else:
23         print("Não foi possível obter a cotação.")
24
25 # Programa principal (procedural)
26 dados = buscar_cotacoes("USD")
27 cotacao_brl = pegar_cotacao(dados, "BRL")
28 exibir_cotacao(cotacao_brl, "BRL")
```

Exemplo 2: Verificação de Divisibilidade

```
1 #include <stdio.h>
2
3 // Funcao para obter entrada do usuario
4 int obterNumero() {
5     int num;
6     printf("Digite um numero inteiro: ");
7     scanf("%d", &num);
8     return num;
9 }
10
11 // Funcao para verificar divisibilidade por 3
12 int isDivisivelPor3(int n) {
13     return (n % 3 == 0);
14 }
15
16 int main() {
17     int numero = obterNumero();
18
19     if (isDivisivelPor3(numero)) {
20         printf("%d e divisivel por 3\n", numero);
21     } else {
22         printf("%d nao e divisivel por 3\n", numero);
23     }
24
25     return 0;
26 }
```


Outputs:

17 / 20

Referências I

-  IBM. John Backus. IBM History, 2024. Disponível em: <https://www.ibm.com/history/john-backus>. Acesso em: 30 out. 2025.
-  Wired. Dennis Ritchie, criador do C e do Unix. Wired Magazine, 2011. Disponível em: <https://www.wired.com/2011/10/dennis-ritchie/>. Acesso em: 30 out. 2025.
-  Communications of the ACM. In Memoriam: Niklaus Wirth. ACM, 2024. Disponível em: <https://cacm.acm.org/news/in-memoriam-niklaus-wirth/>. Acesso em: 30 out. 2025.
-  Google LLC. Website oficial. Disponível em: <https://about.google/>. Acesso em: 30 out. 2025.
-  Microsoft Corporation. Website oficial. Disponível em: <https://www.microsoft.com/>. Acesso em: 30 out. 2025.
-  Amazon.com, Inc. Website oficial. Disponível em: <https://www.aboutamazon.com/>. Acesso em: 30 out. 2025.
-  Apple Inc. Website oficial. Disponível em: <https://www.apple.com/>. Acesso em: 30 out. 2025.
-  Guia Dev. Paradigmas de Programação. Disponível em: <https://guia.dev/pt/pillars/languages-and-tools/programming-paradigms.html>. Acesso em: 30 out. 2025.
-  Rocketseat. Introdução aos Paradigmas de Programação. Disponível em: <https://www.rocketseat.com.br/blog/artigos/post/introducao-aos-paradigmas-de-programacao>. Acesso em: 30 out. 2025.

Quizz

B de Bis:

<https://wayground.com/join?gc=665133>

Código: 665133

